LifeLink: A Smart Blood Donation and Emergency Request Management System

Harshavardhini M., Sandhiya E., and Thamizhini R.
Department of Computer Science and Design, Rajalakshmi Engineering College, Chennai, Tamil Nadu, India
Under the guidance of Dr. P. Revathy

***Abstract*—Obtaining a suitable blood donor during an urgent medical situation is a coordination problem as much as an availability problem. Information about potential donors, hospitals, blood banks and request status is often scattered across phone contacts, informal messages and separate sources. This paper presents LifeLink, a Flutter and Firebase mobile application that brings these activities into one request-centred workflow. The application provides user authentication, blood-request creation and management, blood-group-based donor discovery, hospital and blood-bank search, previous donor history, priority donor suggestions, request acceptance with notifications, and two-way live location sharing between a patient and an accepted donor. Requests and participant locations are stored as cloud records, so that both sides can retrieve the latest reported positions during coordination. The priority suggestion uses recorded donation count as a transparent ordering signal; it does not establish medical eligibility, current availability or willingness to donate, which must be confirmed by qualified healthcare personnel. Location sharing depends on device permission, GPS, connectivity and successful cloud updates, and therefore shows the latest received position rather than guaranteed continuous tracking. The paper describes the motivation, architecture, functional modules, data organization, privacy considerations and a functional verification plan. No measured reduction in donor response time or clinical benefit is claimed; such conclusions require a controlled evaluation.**

***Keywords*—blood donation, emergency request management, donor discovery, donor history, live location tracking, Flutter, Firebase, healthcare information system**

I. Introduction

Blood transfusion services depend on the availability of safe blood and on the systems used to collect, test, store and distribute it. At the level of an individual patient or attendant, however, finding a donor involves a separate set of coordination tasks. The requester must communicate the required blood group, identify a suitable hospital or blood bank, contact potential donors and follow up on their responses. When these details are spread across different channels, the requester has to repeat information and track the progress of the request manually.

LifeLink was developed as a mobile coordination application for this practical problem. It places the blood request at the centre of the user workflow and connects it with donor discovery, hospital and blood-bank search, donor history, request status, notifications and participant location sharing. The application is a coordination aid. It is not intended to replace medical assessment, licensed blood-bank operations or institutional transfusion procedures.

A. Motivation

A requester needs more than a list of names. The requester must identify the required blood group, understand which recorded donors may be relevant, associate the request with a healthcare facility and know whether a donor has responded. Once a request is accepted, the two participants may also need to coordinate their whereabouts. LifeLink combines these related tasks in a single application so that request information and communication remain connected.

B. Objectives and Contributions

The work makes the following implementation contributions:

- An integrated mobile workflow for creating, managing and viewing blood requests.
- Blood-group-based discovery of recorded donors relevant to a request.
- Hospital and blood-bank search that supplies healthcare-facility context.
- Display of previous donor information and a transparent priority suggestion based on recorded donation history.
- Request acceptance and notification-based communication between participants.
- Two-way live location sharing between the patient and the accepted donor, associated with a single request.
- Cloud-backed storage of application records using Firebase services.

II. Problem Statement and Research Gap

A. Problem Statement

Informal blood-search practices often depend on personal contacts, social-media groups, manually maintained donor lists and direct hospital enquiries. These channels can be useful, but they do not necessarily provide one shared view of the blood requirement, relevant donor information, the donor's response and the participants' locations. A mobile system should therefore organize these activities around a request, while ensuring that users are not misled into treating a donor record as proof of medical suitability or present availability.

B. Research Gap

A donor directory addresses only one part of the coordination process. A requester may still need to search for a healthcare facility, contact several potential donors, determine whether a request has been accepted and coordinate with the responding donor. LifeLink addresses this workflow-level gap by combining donor discovery, previous donor information, facility search, request handling, notifications and two-way location sharing. The contribution lies in the integration of these functions in one mobile workflow, not in a claim that the application replaces established blood-service infrastructure.

III. Related Work and Technical Context

A. Blood Services and Digital Platforms

Blood safety guidance emphasizes that access to blood must be supported by organized blood services and by appropriate quality and safety procedures [1]. In India, e-RaktKosh provides a national digital platform for blood-bank information and blood availability [2]. Such institutional resources play an important role in the wider blood-service ecosystem. LifeLink has a different and complementary focus: it organizes requester-to-donor communication and request-related interaction at the level of a mobile application.

B. Mobile and Cloud Technologies

Mobile application frameworks and cloud backend services make it possible to build applications whose interfaces communicate with centralized data services. Flutter provides a cross-platform framework for mobile development [3]. Firebase Authentication supports identity-related workflows, and Cloud Firestore provides a cloud-hosted database for storing and retrieving application records [4], [5]. Firebase Cloud Messaging supports notification delivery [7]. These technologies form the implementation basis for LifeLink.

C. Location-Aware Coordination

Location-aware applications can obtain device coordinates, for example through a Flutter geolocation plugin [8], and share them with authorized users, and an OpenStreetMap-based map [9] can display them. In LifeLink, the location feature is bound to one blood request and its two participants. This matters because location information is sensitive: the application should restrict access to the relevant patient and accepted donor, request only the permissions it needs and avoid exposing coordinates to unrelated users. Firebase documentation describes security rules that developers can use to enforce such access conditions on stored data [6].

IV. System Architecture

A. Architectural Overview

LifeLink follows a mobile-client and cloud-backend architecture, illustrated in Fig. 1. The Flutter application is the presentation and interaction layer: it collects user input, displays donor and facility information, submits blood requests and presents the tracking map. Its functions are organized into application modules (discovery, search, history and priority suggestion) and request-coordination functions (creation, acceptance, notifications and status). Firebase Authentication supports sign-in. Cloud Firestore stores application records, including user information, donor records and blood requests. Request-associated live location records are kept as separate records linked to the relevant request, so that the latest position of each participant can be retrieved in the context of that request.

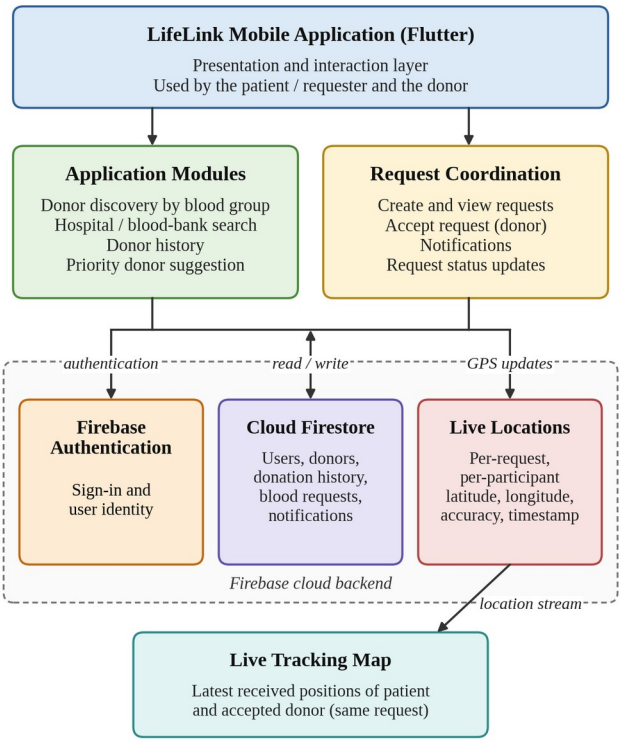


Fig. 1. High-level architecture of LifeLink: the Flutter client, Firebase services, application modules, request records and participant location updates.

B. Major Components

- **User and authentication module:** supports sign-in and identifies the current application user.
- **Blood request module:** captures the blood group and other request details and maintains request status.
- **Donor discovery module:** presents donor records relevant to the selected blood group.
- **Hospital and blood-bank search module:** provides facility-related search information.
- **Donor history module:** displays previously recorded donation information.
- **Priority suggestion module:** orders potential donor contacts using recorded donation count as one ranking signal.
- **Request and notification module:** supports donor response and request-related status communication.
- **Live location module:** obtains participant coordinates and stores updates against the associated request.
- **Cloud data module:** provides persistence and retrieval of application information through Firebase.

C. Request-Centred Data Flow

A requester creates a blood request through the mobile interface, and the application writes it to the cloud database, from where the relevant screens retrieve it. A donor can review an available request and submit an acceptance response; the request record is then updated with the response and the donor associated with it, and a notification informs the relevant participant. When the patient and the accepted donor use the tracking feature, each participant's location update is stored against the same request identifier. The tracking interface retrieves the latest available location records and displays them on a map. The shared request

record is therefore the single reference point for the whole interaction.

V. Functional Modules and Workflow

A. Blood Request Creation and Management

The requester supplies the information that describes the blood need, including the required blood group, the associated hospital and the number of units. The request is stored as a cloud record so that it can be retrieved and displayed across sessions. Request status and timestamps distinguish an active request from one that has been updated.

B. Blood-Group-Based Donor Discovery

Discovery uses the blood group selected for the request to identify relevant donor records, narrowing the displayed set to people whose recorded blood group matches the search criterion. The feature supports contact discovery only. Actual compatibility, eligibility and suitability must be confirmed by qualified healthcare personnel and the relevant blood bank.

C. Previous Donor History and Priority Suggestions

LifeLink displays previous donor information and uses donation count as a simple prioritization signal. Let D_g denote the set of recorded donors with blood group g, and let $H(d)$ denote the number of donations recorded for donor d. The conceptual ordering is

$$\text{Priority}(d) \propto H(d), \quad d \in D_g \tag{1}$$

Donors with a higher recorded donation count are displayed earlier in the suggested list. The rule is deliberately simple and explainable. It does not establish that a donor is currently available, willing, medically eligible or due for another donation, and its accuracy depends on the completeness and correctness of the recorded history. Any future enhancement should use verified information and be reviewed against appropriate medical guidance.

D. Hospital and Blood-Bank Search

The search feature lets users look up healthcare-facility information available through the application. It gives context to the request and helps users identify the facility associated with the blood requirement. Facility listings are informational; users should confirm stock, operating procedures and availability directly with the institution.

E. Request Acceptance and Notifications

A donor can respond to a blood request through the application. On acceptance, the application updates the request record with the response status and donor details. Request-related notifications inform the relevant participant of updates, and the shared request record acts as the reference point for the interaction.

F. Two-Way Live Location Tracking

LifeLink supports location sharing by both participants of an accepted request. After the user grants the required permission and enables device location services, the application obtains a device position and writes it to the live-location area of the blood request. A location record can contain the participant identifier, request identifier, latitude, longitude, reported accuracy, update timestamp and an activity indicator. The other participant retrieves the latest available record and sees it on the map.

The feature supports coordination between patient and donor. It should not be described as guaranteed uninterrupted tracking: updates depend on GPS reception, device settings, permission state, connectivity and successful database operations. The displayed marker is the latest location received by the application and may become stale if updates stop. Access to location records must be restricted to the participants of the corresponding request.

VI. Data Organization and Implementation

A. Flutter Application Layer

Flutter implements the mobile interface and connects the application screens to the backend services. The interface provides the user-facing workflows for blood requests, donor information, hospital and blood-bank search, notifications and live tracking. A single mobile application layer allows the different modules to share the same request context.

B. Firebase Services

Firebase Authentication provides application identity and sign-in. Cloud Firestore stores user records, donor information and blood requests. A request record contains the information the application uses to identify the requester, required blood group, associated hospital, request status and relevant timestamps. Live location updates are stored under the corresponding request, separated by participant identifier, which makes it possible to retrieve the latest location of each participant in the context of the active request.

C. Main Data Entities

The principal entities handled by the application are summarized in Table I.

TABLE I
MAIN DATA ENTITIES

Entity	Content
User	Authenticated account and profile attributes.
Blood Request	Requester identifier, blood group, hospital, units, status and timestamps.
Donor	Donor identity and blood-group-related information.
Donation History	Recorded donation count or history used for display and prioritization.
Hospital / Blood Bank	Facility information available through the application.
Live Location	Request identifier, participant identifier, coordinates, accuracy, update timestamp and activity state.
Notification	Request-related message or status update intended for a user.

D. Location Update Procedure

When a participant opens the tracking function, the application reads the corresponding request and checks whether the signed-in account is one of its participants. It then requests the device position. If a valid position is available, the application writes the first location record and subscribes to subsequent position updates. Each update is stored against the request and participant, and the tracking screen listens for the corresponding records and refreshes the map when new values arrive. If permission, GPS or

database operations fail, the application should report the problem rather than present an old location as current.

E. Security and Privacy Considerations

Location and request information can reveal sensitive personal circumstances. The system should apply least-privilege access, restrict request documents and location records to authorized participants, and avoid collecting location when the user has not enabled the tracking workflow. Authentication alone is not sufficient: database security rules must also validate the relationship between the signed-in user and the request [6]. Both permitted and denied access cases should be tested before deployment. Users should be informed when location sharing is active and should have a clear way to stop sharing. Donor records should be limited to the information needed for discovery, and any personal or location data used for testing should be minimized and collected with consent.

VII. Functional Verification and Evaluation Plan

Functional verification is carried out on the deployed application and the corresponding cloud records. Table II lists the checks and the observable behavior expected for each. The result column is reserved for the team's recorded test evidence; it must be completed only from actual test logs and never with assumed or estimated outcomes.

TABLE II
FUNCTIONAL VERIFICATION CHECKLIST

No.	Test Area	Verification Procedure	Expected Observable Behavior	Recorded Result
1	Authentication	Sign in with an authorized account.	Access to the application is granted.	
2	Request creation	Submit a blood request with the required details.	Stored request fields match the submitted input.	
3	Donor discovery	Select a blood group and inspect the displayed donor set.	Displayed donors match the selected blood group.	
4	Donor history	Compare displayed history with the stored donor records.	Displayed history matches the stored records.	
5	Priority suggestion	Verify that the configured history signal affects ordering.	Donors with a higher recorded donation count appear earlier.	
6	Facility search	Search for a hospital or blood bank and inspect the returned details.	Returned facility details are displayed correctly.	
7	Request acceptance	Accept a request from a donor account.	Request status and donor identity are updated.	
8	Notifications	Check delivery and the corresponding request update.	The relevant participant is notified of the update.	
9	Patient location	Share location from the patient account.	Patient coordinates and timestamp are stored against the request.	
10	Donor location	Share location from the accepted donor account.	Donor coordinates and timestamp are stored against the request.	
11	Two-way tracking	Open the tracking screen on both accounts.	Each participant sees the other's latest received position.	
12	Access control	Try to read request locations from an unrelated account.	Access is denied.	

A. Suggested Quantitative Evaluation

A later evaluation can measure the time needed to create a request, the number of interactions needed to reach the donor discovery screen, the delay between a location update and its appearance on the other participant's map, and the success rate of request and location retrieval. Each measurement should state the device, network condition, number of trials and calculation method. For example, location-update delay can be computed as the difference between the time an update is generated and the time the receiving client observes it. Averages and variation should be reported only after repeatable measurements have been collected.

B. User-Based Evaluation

Representative patients or attendants and donors can be asked to complete realistic tasks with the application. Task completion, time taken, navigation errors, perceived clarity and confidence in request status can be recorded. Participants should be told the purpose of the evaluation, and any personal or location information collected should be minimized and handled with consent.

VIII. Results and Discussion

The implemented application brings the main LifeLink workflows into one mobile application: blood-request creation, blood-group-based donor discovery, hospital and blood-bank search, previous donor information, priority suggestions, request acceptance, notifications and two-way live location sharing. The integration gives the requester and the donor a shared request context instead of treating each activity as an unrelated feature.

The donor-history ranking rule is easy to understand, but its usefulness depends on accurate records, and it must not be read as medical advice. Likewise, location sharing provides useful coordination context, but the freshness of a displayed position depends on the update path from the device to the

cloud and back to the other participant. Error states, missing permissions and stale updates must therefore be communicated clearly to the user.

This paper reports the functional scope and implementation design. It does not claim a measured reduction in donor search time, an increase in successful donations or improved clinical outcomes, because no controlled evaluation data have been collected. Such conclusions would require documented experiments and real user observations.

IX. Advantages and Limitations

A. Advantages

- Integrates blood requests and donor discovery in one mobile workflow.
- Blood-group-based filtering makes donor discovery more focused.
- Displays previous donor information with an explainable priority signal.
- Connects donor discovery with hospital and blood-bank search.
- Supports request acceptance and notification-based updates.
- Lets patient and donor view the latest locations shared for their request.
- Uses cloud-backed records that can be retrieved across sessions.

B. Limitations

- Donation history alone cannot determine present availability or medical eligibility; eligibility must be verified by qualified healthcare personnel.
- Incorrect or outdated donor and facility records reduce usefulness.
- Location updates depend on GPS, permissions, connectivity and backend availability.
- Location data require consent, restricted access and secure handling.
- The donor-priority approach uses a single, limited ranking signal.
- Real-world effectiveness and usability have not yet been established by a controlled study.
- The application does not replace licensed blood-bank processes or clinical screening.

X. Future Work

- Introduce verified donor availability and last-donation information under suitable medical guidance.
- Add distance-aware donor suggestions with privacy-preserving location controls.
- Provide clear controls for starting and stopping location sharing.
- Strengthen role-based and request-participant-based database access rules.
- Improve verification of hospital and blood-bank information.
- Measure workflow usability, location-update delay and cloud data consistency.
- Conduct a controlled user study with documented participant consent and repeatable tasks.
- Explore privacy-conscious analytics of request status and response patterns.

XI. Conclusion

This paper presented LifeLink, a Flutter and Firebase mobile application that organizes blood-request coordination. It combines blood-request creation and management, blood-group-based donor discovery, hospital and blood-bank search, previous donor history, priority donor suggestions, request acceptance, notifications and two-way live location sharing between a patient and an accepted donor under a common request workflow.

The main contribution is this integration of donor discovery and request coordination with participant location sharing. The platform is intended to support communication and access to relevant information; medical eligibility and final donation decisions remain within established healthcare procedures. Further work should evaluate usability, location-update reliability, data consistency, security and practical effectiveness using measured results.

Acknowledgment

The authors thank Dr. P. Revathy for her guidance and the Department of Computer Science and Design, Rajalakshmi Engineering College, Chennai, for its support.

References

[1] World Health Organization, “Blood safety and availability,” Fact sheet, WHO, Geneva, Switzerland. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability. Accessed: Nov. 2, 2026.

[2] National Health Mission, Government of India, “e-RaktKosh: Centralized blood bank management system.” [Online]. Available: https://www.eraktkosh.in/. Accessed: Nov. 2, 2026.

[3] Google, “Flutter documentation.” [Online]. Available: https://docs.flutter.dev/. Accessed: Nov. 2, 2026.

[4] Google, “Firebase Authentication documentation.” [Online]. Available: https://firebase.google.com/docs/auth. Accessed: Nov. 2, 2026.

[5] Google, “Cloud Firestore documentation.” [Online]. Available: https://firebase.google.com/docs/firestore. Accessed: Nov. 2, 2026.

[6] Google, “Get started with Cloud Firestore Security Rules,” Firebase documentation. [Online]. Available: https://firebase.google.com/docs/firestore/security/get-started. Accessed: Nov. 2, 2026.

[7] Google, “Firebase Cloud Messaging documentation.” [Online]. Available: https://firebase.google.com/docs/cloud-messaging. Accessed: Nov. 2, 2026.

[8] Geolocator package contributors, “geolocator: Flutter geolocation plugin,” pub.dev. [Online]. Available: https://pub.dev/packages/geolocator. Accessed: Nov. 2, 2026.

[9] OpenStreetMap contributors, “OpenStreetMap.” [Online]. Available: https://www.openstreetmap.org/. Accessed: Nov. 2, 2026.